TripMate AI on AWS - Solution Architecture

Which AWS services power the app, and why

Application: TripMate AI - a multi-agent travel planner (7 AI agents, MCP tools, Responsible-AI guardrails) for trips worldwide. **This document:** describes the app running on **AWS**, naming the service used for each part of the system and the reason it was chosen.

---

1. Overview

TripMate AI is deployed on AWS as a managed, secure, scalable service. A team of AI agents - a Supervisor plus five specialists (Flight, Hotel, Weather, Budget, Itinerary) and a Final Response composer - is orchestrated with **Google ADK**, runs on **Amazon Bedrock AgentCore**, is powered by foundation models on **Amazon Bedrock**, and is protected end-to-end by **Amazon Bedrock Guardrails**. Knowledge is grounded with **Bedrock Knowledge Bases** and **Amazon Neptune** (GraphRAG), and quality is measured with **Bedrock Evaluations**.

---

2. AWS services at a glance

Layer	AWS service	Role in TripMate AI
Foundation models (LLM)	**Amazon Bedrock**	The reasoning engine for all 7 agents
Agent runtime & ops	**Amazon Bedrock AgentCore** (Runtime, Memory, Gateway, Identity, Observability)	Hosts and operates the agent workflow
Safety / Responsible AI	**Amazon Bedrock Guardrails**	Content safety, PII redaction, denied topics, grounding
Knowledge / RAG	**Bedrock Knowledge Bases** + **Amazon OpenSearch Serverless**	Retrieval-augmented grounding of answers
GraphRAG	**Amazon Neptune (Analytics)**	Relationship-aware retrieval (place > attractions > routes)
Tool functions	**AWS Lambda** (behind AgentCore Gateway)	Weather, flights, search, places tools as MCP
Authentication	**Amazon Cognito**	User login/registration, MFA, tokens
Content & assets	**Amazon S3**	Knowledge-base source docs, static files
Secrets	**AWS Secrets Manager**	Third-party API keys (weather, flights, search)
Observability	**AgentCore Observability + Amazon CloudWatch + AWS X-Ray**	Tracing, logs, metrics, dashboards
Evaluation	**Amazon Bedrock Evaluations**	Quality, groundedness and safety scoring
UI / API hosting	**AWS App Runner** (or **ECS Fargate**) + **Amazon API Gateway**	Serves the Streamlit UI and REST API
Security & networking	**IAM, VPC, AWS KMS**	Access control, isolation, encryption

---

3. Architecture

---

4. Service-by-service - what it does here and why

4.1 Amazon Bedrock - the LLM for every agent

- **Used for:** powers the Supervisor and all specialist agents (Flight, Hotel, Weather, Budget, Itinerary) and the Final Response agent. A strong reasoning model (e.g. **Anthropic Claude**) is used for the Supervisor and Itinerary agents; a cost-efficient model (e.g. **Amazon Nova** or **Llama 3**) for simpler summarisation agents.
- **Why Bedrock:** one API for multiple top models, no GPU to manage, elastic scale, data isn't used to train the models, and region/compliance control - ideal for a production agent system.

4.2 Amazon Bedrock AgentCore - runtime & operations

- **AgentCore Runtime** - hosts the **Google ADK** workflow serverlessly (AgentCore is framework-agnostic, so the existing graph is reused, not rewritten). *Why:* no servers to run, scales per request, managed sessions.
- **AgentCore Memory** - stores each user's long-term preferences (e.g. "vegetarian", "prefers trains") and conversation history. *Why:* managed, durable memory without operating a database.
- **AgentCore Gateway** - exposes the tool functions as governed **MCP** endpoints. *Why:* the app already speaks MCP; Gateway adds auth, throttling and logging centrally.
- **AgentCore Identity & Observability** - secure agent identity and full tracing of multi-agent runs. *Why:* visibility and control required for production.

4.3 Amazon Bedrock Guardrails - Responsible AI at every step

- **Used for:** the input guardrail and the output of every agent - content safety filters, **PII detection & masking** (email, phone, Aadhaar, PAN, cards), **denied topics** (enforces the worldwide policy), prompt-attack filtering, and **contextual grounding** checks on the final plan.
- **Why Guardrails:** one managed, versioned, auditable policy enforced consistently across all agents - less custom safety code and a clear compliance story.

4.4 Bedrock Knowledge Bases + OpenSearch Serverless - RAG

- **Used for:** grounding the Hotel, Itinerary and Final agents in curated India travel content (city guides, seasons, transport, attractions) stored in **S3** and indexed as vectors in **OpenSearch Serverless**.
- **Why:** reduces hallucination and keeps recommendations accurate and current.

4.5 Amazon Neptune - GraphRAG

- **Used for:** relationship-aware retrieval - *destination > top attractions > nearby places > how to get there* - so itineraries group sights sensibly and suggest realistic routes.
- **Why:** some travel questions are about connections between entities, which a graph answers better than plain vector search.

4.6 AWS Lambda - tool functions (behind AgentCore Gateway)

- **Used for:** the weather, flights, search and places tools (wrapping Open-Meteo, AviationStack, Tavily and Google Places) run as Lambda functions exposed via Gateway as MCP tools.
- **Why:** serverless, pay-per-call, independently scalable and secured; keeps the specialist data providers while governing access.

4.7 Amazon Cognito - authentication

- **Used for:** user registration, login, MFA and token-based sessions (replacing the local SQLite login).
- **Why:** managed, secure identity that scales to many users.

4.8 Amazon S3 - content & assets

- **Used for:** source documents for the knowledge base and any static assets.
- **Why:** durable, cheap, integrates directly with Bedrock Knowledge Bases.

4.9 AWS Secrets Manager - third-party keys

- **Used for:** securely storing API keys for the external travel data providers.
- **Why:** no secrets in code or env files; rotation and access control built in.

4.10 Observability - AgentCore Observability + CloudWatch + X-Ray

- **Used for:** tracing each agent run, logs, latency/cost metrics and dashboards/alarms.
- **Why:** operate the system with confidence and debug multi-agent behaviour.

4.11 Amazon Bedrock Evaluations - quality & safety

- **Used for:** scoring itinerary quality, answer groundedness (RAG), and guardrail/PII effectiveness, with LLM-as-a-judge and regression runs before each release.
- **Why:** objective evidence that changes improve (and don't regress) quality and safety.

4.12 Hosting - App Runner / ECS Fargate + API Gateway

- **Used for:** serving the **Streamlit** UI and the REST API.
- **Why:** managed container hosting with autoscaling; API Gateway adds throttling and auth.

4.13 Security & networking - IAM, VPC, KMS

- **Used for:** least-privilege access (IAM), network isolation (VPC endpoints to Bedrock/OpenSearch), and encryption at rest/in transit (KMS).

- **Why:** enterprise security and compliance baseline.

---

5. Request flow on AWS (end to end)

1. A user signs in via **Cognito** and submits a request through the **App Runner**-hosted UI / **API Gateway**.
2. **AgentCore Runtime** starts the **Google ADK** workflow.
3. The **Input Guardrail** runs **Bedrock Guardrails** (safety, PII, denied topics, prompt-attack).
4. The **Supervisor** (a **Bedrock** model) extracts trip details, enforces the worldwide policy, and picks the agents to run.
5. Optional **human review** approves the plan.
6. The selected **specialists** run in parallel on **Bedrock**, calling **Lambda** tools via **AgentCore Gateway** (MCP) and retrieving grounded facts from **Knowledge Bases / Neptune**.
7. The **Final** agent composes the plan; **Bedrock Guardrails** run a final safety + PII pass.
8. State is saved to **AgentCore Memory**; traces go to **Observability/CloudWatch**; the plan is returned.
9. Offline, **Bedrock Evaluations** score quality and safety for continuous improvement.

---

6. Why AWS for this application

- **Managed & scalable:** no GPUs, databases or servers to operate; scales from one user to many.
- **Responsible AI built in:** Bedrock Guardrails enforce safety, privacy and scope consistently and auditably across every agent.
- **Accurate & grounded:** Knowledge Bases + Neptune reduce hallucination and keep advice realistic.
- **Secure & compliant:** IAM, VPC, KMS, Cognito and Secrets Manager provide an enterprise security baseline; data is not used to train the models.
- **Measurable:** Observability + Evaluations give the metrics needed to run the service with confidence.
- **Architecture reuse:** the multi-agent + MCP + guardrail design maps directly onto AWS - AWS mostly provides managed backends for the same components.

---

7. Service summary

Component	AWS service
LLM for 7 agents	Amazon Bedrock
Agent hosting / memory / tool gateway / identity / tracing	Amazon Bedrock AgentCore
Responsible AI (safety, PII, scope, grounding)	Amazon Bedrock Guardrails
RAG knowledge	Bedrock Knowledge Bases + OpenSearch Serverless
GraphRAG	Amazon Neptune
Tool functions (weather/flights/search/places)	AWS Lambda
Authentication	Amazon Cognito
Content storage	Amazon S3
Secrets	AWS Secrets Manager
Observability	CloudWatch + X-Ray + AgentCore Observability
Evaluation	Amazon Bedrock Evaluations
UI / API hosting	AWS App Runner / ECS Fargate + API Gateway
Security & networking	IAM, VPC, KMS

TripMate AI on AWS - a managed, secure, scalable multi-agent travel planner.